

School of Computing, Engineering, and Physical Sciences
Assessment Guidance Coversheet

Module Code:	COMP10037	
Module Title:	Games Console Programming	
Module Co-ordinator:	Dr. Paul Keir	
Assessment set by:	Dr. Paul Keir	
Learning Outcomes Assessed:	L1 Demonstrate a critical appreciation of the hardware architectures of modern games console platforms. L2 Apply design techniques and optimization strategies for developing games software for a modern games console platform using a low-level API. L3 Demonstrate a critical appreciation of the differences between coding for general purpose platforms (e.g. PC) and fixed hardware platforms with limited OS services. L4 Demonstrate critical awareness of the issues surrounding parallel processing, and its use on modern games console platforms.	
Issued:	Tuesday, October 6th, 2026	
Deadline:	5pm, Friday, October 23rd, 2026	
Deadline:	5pm, Friday, November 13th, 2026	
Individual / Group assessment:	Individual ☐	Group ☒

Anonymity

The University Regulations makes it clear that all assessments are marked anonymously (Regulation 3.4) unless the assessment itself renders anonymity impossible e.g. placements, presentations, practical assessments. Unless otherwise stated, do not put your name or Banner ID on your submission.

Referencing

The standard referencing style at UWS is **Cite Them Right (CTR) Harvard** and your references should be in accordance with these guidelines. The University provides a drop-down menu that lets you see examples of how you should reference journals, books, websites etc. Guidance on referencing styles can be found on the UWS Library website: <https://uws-uk.libguides.com/referencing>.

Extenuating Circumstances

The University recognises that, from time to time, you may encounter issues which may prevent you from being able to submit or undertake an assessment. Where this is the case, you can complete an Extenuating Circumstances Submission (ECS) for consideration. The ECS will be forwarded to the School Assessment Board to take account of this declaration in recording your module marks¹. Guidance on ECS

¹Note that any mark / grade you receive is provisional and may be subject to change until there has been internal

claims can be found: <https://www.uws.ac.uk/current-students/supporting-your-studies/exams-assessment-appeals/academic-appeals-extenuating-circumstances>.

Generative AI

The type of Generative AI you are allowed to use within your assessment is:

Type	Description	Allowed
1	Restricted Only the use of routine and established tools, such as auto-transcription, spell checkers, grammar check is permitted.	☒
2	Specified Generative AI may be used for clearly delineated tasks as appropriate / allowed / recommended, although its use is not mandatory in order to complete the assessment.	☐
3	Open No specific restrictions but with requirement to track key stages / tools utilised. The use of such tools is not mandatory in order to complete the assessment. This may include: (i) Socratic chatbot, (ii) summarisation, simplification, synthesis, and translation, (iii) generative illustrative media content, and (iv) support production of multimedia artefacts.	☐
4	Embedded Generative AI is a feature of the assessment itself. Here the use of Generative AI is a focal aspect of the assessment. This may include (i) using named tools for specific outcome, (ii) output comparison and critique, and (iii) develop or error check code.	☐

Assessment Student Declaration Coversheet

All students are expected to complete the Student Declaration and insert that as the first page of your submission. The Module Co-ordinator will have uploaded this to Aula for you to complete.

- For written assessments, insert the Declaration as the first page of your document.
- For assessments not in the written format (e.g. video, audio, presentation, or practical work), submit the Declaration as a separate file.
- For group assessments, unless otherwise directed by your lecturer, the group may submit a single shared declaration.

moderation, external examination and ratification at the School Assessment Board.

First Assignment

Games Console Programming

COMP10037

Issue Date: Tuesday, October 6th, 2026
Due Date: **5pm, Friday, October 23rd, 2026**

3D Model Interaction on the PlayStation 5



In this assignment you will develop an impressive interactive graphical application for the PS5TM by adding features to the program provided. The picture above is of a finished version of the program.

You should work in your allocated pair. Submit the code you develop, along with a short pdf report of approximately 1000 words; as a single zip to Brightspace by the date noted above. Submit your Visual Studio project, and any necessary resources. Please do not submit the executable or intermediate ob-

ject files (I will always build it myself). Choose “Clean Solution” before zipping your submission.

Resources

Unzip the assignment, and use CMake to configure Visual Studio from the `CMakeLists.txt` provided, as you did in the labs: choose your version of Visual Studio as the generator, select *Specify toolchain file for cross-compiling*, and give it the PS5’s toolchain file:

`C:/Program Files (x86)/SCE/Prospero/Tools/CMake/PS5.cmake`

Then *Generate*, and *Open Project*. Building and running it shows a Teenage Mutant Ninja Turtle, lit by a single light, playing the first of the five animations stored in its model file. The right analogue stick swings the camera around him; Circle quits.

Inside the zip you will find:

- **src**: the program, in three files:
 - **main.cpp** *makes* things: the window, the graphics device, the shaders, the buffers, the model. Each is created once, checked once, and handed to the **Game**.
 - **game.hpp** and **game.cpp** *do* things with them. Nearly everything you add belongs in one of **Game**’s four functions: **process_events** (what the player did), **update** (what has changed because of it), **interface** (what the ImGui panel should say) and **render** (what the screen should show).
- **shaders**: **model.vert** and **model.frag**, the GLSL that runs on the GPU. CMake compiles these to SPIR-V when you build, so a shader you’ve edited takes effect only after building again.
- **resources**: the model; plus the music & sound effect from the week 2 lab.
- **deps**: SDL3 for the PS5, the ZOD library, Dear ImGui, GLM, and TinyGLTF.

Dear ImGui is already started, and already draws an empty panel; the model is already loaded, animated and lit. Everything below is a change to a program that runs.

The lectures and labs of weeks 1 to 4 are the reference material for the assignment. Comments in the code should also be useful.

Methodology

The program you are provided with displays an animated, lit model, seen from a camera you can swing around it. You should aim to develop it further into a more interactive graphical and aural form. You will be rewarded for using your imagination as well as your programming ability. To provide the necessary mark and grading structure, a set of requirements is provided.

Requirements

A set of improvements to the original program are requested:

1. **Translate the camera with the left analogue stick.** Read it with `pads_.stick(Stick::Left)`, which gives a `glm::vec2` with the dead zone already taken out, and move the camera with `camera_.move_local`. That member function moves the camera in the directions it is facing rather than the world's, and moves the target with the eye, so pushing away from you slides the model across the view. Multiply by `dt` so the speed does not depend on the frame rate, and by `radius_` so that it suits the size of the model.
2. **Move the camera towards and away from the model with the triggers.** The triggers are axes rather than buttons (`pads_.axis` with `SDL_GAMEPAD_AXIS_LEFT_TRIGGER` and `SDL_GAMEPAD_AXIS_RIGHT_TRIGGER`) so they read 0 when loose and 1 when pulled all the way. Use `camera_.move_position`, which moves the eye and leaves the target where it is; that is what “closer” has to mean. Stop short of the target: a camera standing on the thing it is looking at has no direction left to look in, and the screen fills with nothing.
3. **Stop and start the animation with a button.** The animation is read from `time_`, which `update` advances every frame. Add a `bool` member, and advance `time_` only when it is true. Note what does *not* stop: the model is still drawn, still lit, and the camera still moves, so you can still move around a pose and inspect it.
4. **Switch between the model's animations with a button.** The model file holds five. `model_.animation_count()` says how many, and `animation_` says which one `render` asks for; set `time_` back to 0 as you change it, so the new cycle starts at its beginning. Use `pads_.pressed(...)`, which is true on the frame a button goes down, rather than `pads_.held(...)`, which would change the animation sixty times a second.
5. **Play a sound effect when the animation is switched.** `resources/audio/sfx` holds the sound effect from the week 2 lab. As in that lab: load it with `Wav::create`, open an `AudioDevice`, and make an `AudioStream` to play it on. Put the samples into the stream with `SDL_PutAudioStreamData` and follow them with `SDL_FlushAudioStream`. Press the button twice quickly and you will see why `SDL_ClearAudioStream` is wanted first.
6. **Play background music.** The piece is in `resources/audio/music`. It needs a second `AudioStream` bound to the same `AudioDevice`, which mixes the two (one stream cannot play both). Queue the whole piece at once. To make it repeat, ask `SDL_GetAudioStreamQueued` how much is left and queue it again when there is nothing; `SDL_SetAudioStreamGain` will keep its volume under the sound effect rather than over it.

7. **Add another light, of a different colour and in a different position.**
The lights are a `zod::PointLight` array in `game.hpp`, placed in `Game`'s constructor and pushed to the fragment shader in `render_model.frag` declares an array of the same length and loops over it. Change `light_count` in both places: the C++ pushes exactly what it says, and a shader expecting more than it is sent reads bytes that were never written - which goes wrong quietly, rather than failing to build. Choose a colour and a position that can be told apart from the first light's on the model. More than one extra light is welcome, and so is a light that moves.
8. **Show statistics about the model with Dear ImGui.** The empty panel in `Game::interface` needs some content: `ImGui::Text` takes a format string, as `printf` does. The model knows most of what you might say about it - `model_.indices().size() / 3` is the number of triangles, `model_.animation_count()` the number of animations, and `model_.animation_name(animation_)` the name of the one playing. The file's name is not something the model keeps, so you will have to get it from `main.cpp` to the `Game`; perhaps the constructor could take another argument.
9. **Peer review.** Each of you should comment briefly on the input from your team mate, along with a mark out of 10 for them. (Do not grade yourself.) As this is an individual task, there is another submission point on Brightspace for this. Word or PDF are preferred.

Marking Scheme

The assignment is worth 40% of the marks awarded for the entire COMP10037 module. The following provides a breakdown of the marking scheme:

Report	1000 word report with figures	8
Tasks	1. Camera translation with the left stick	3
	2. Camera movement towards/away with the triggers	2
	3. Stop and start the animation	2
	4. Switch between the model's animations	3
	5. A sound effect when the animation is switched	3
	6. Background music	3
	7. A second light, of another colour and position	5
	7b. A third light, or a light which moves	2
	8. Model statistics shown with Dear ImGui	5
	9. Average of your peer review mark from your team	4
Total		40

The Report

The report is the task worth the most marks. Write it using paragraphs of prose. You can use bulletpoints if required; but do not use them to structure longer sections of text. Say what you changed and why; show a figure or two of

the result; and pick one or two things that went wrong and explain what was actually happening. A paragraph on something you found out that the lectures did not tell you can provide a welcome balance against a purely descriptive section.

Resources You Add

The model you are given is somebody else's work, published under a Creative Commons licence: `credit.md` and the `LICENSES` folder beside it say whose and under what terms, and they must stay with it. If you add resources of your own (a different model; other music; or sound effects) the same applies: say in your report where each came from and under what licence. Week 4's lecture covers what an attribution has to carry.

Plagiarism

Ensure your submission comes only from yourself or your team. You can discuss ideas with your fellow students regarding how to prepare a solution, but the copying or sharing of code is not permitted.

Anonymity

Please use only your Banner ID to identify yourself in your submission. Be aware that version control metadata might also provide your name within Visual Studio.